COMPUTER NETWORKS — PBL PRACTICAL SOLUTIONS

Sanjivani University | Course: 24UAMPC301 | B.Tech AIML

Exam preparation guide based on the 24 questions in the supplied question bank. Use these as worked examples; follow your instructor's device, software, and addressing requirements if they differ.

How to present each practical

Write: Aim → Requirements → Theory → Procedure/Algorithm → Program or Configuration → Output/Verification → Result → Viva. For Packet Tracer, show the topology, IP settings, and successful ping. For code, save with the exact filename, compile, run, and demonstrate a test case.

Q1. OSI Reference Model

Aim: Study the seven OSI layers, their functions, example protocols, and devices.

Requirements: Computer/networking notes; optional Packet Tracer.

Theory: OSI is a conceptual seven-layer model used to describe how data moves between networked systems.

Procedure / Algorithm

1. List the layers from bottom to top: Physical, Data Link, Network, Transport, Session, Presentation, Application.
2. Physical: transmits raw bits over media; examples include cables, radio signals, hubs and repeaters.
3. Data Link: frames, MAC addresses, and local delivery; examples Ethernet (IEEE 802.3), Wi-Fi MAC; switch operates mainly here.
4. Network: logical addressing and routing; examples IPv4, IPv6, ICMP; routers operate here.
5. Transport: process-to-process delivery, ports, reliability and flow control; examples TCP and UDP.
6. Session: establishes, manages, and terminates communication sessions.
7. Presentation: data format, encoding, compression, and encryption; examples include UTF-8, JPEG and TLS-related presentation functions.
8. Application: network services used by applications; examples HTTP/HTTPS, DNS, SMTP, FTP.
9. Explain encapsulation: application data → transport segment/datagram → network packet → data-link frame → bits. Receiving host decapsulates in reverse.

Expected output / verification: Give one protocol/function for every layer and describe a browser request as it travels down and up the stack.

Result: The seven OSI layers and their responsibilities were identified.

Viva: Which layer uses MAC addresses? Data Link. Which uses IP addresses? Network. TCP belongs to which layer? Transport.

Q2. Java TCP Socket Client–Server

Aim: Create a TCP server and client that exchange a message.

Requirements: JDK; terminal or IDE; two terminals on one computer.

Theory: TCP is connection-oriented. Server binds to a port and listens; client connects using host and port.

Procedure / Algorithm

10. Create Server.java and Client.java in the same folder.
11. Start the server first; it waits for a client on port 5000.

12. Run the client in a second terminal and enter a message.
13. The server prints the message and sends a reply.
14. Compile using javac Server.java Client.java; run java Server, then java Client.

Program / Configuration

```
// Server.java
import java.io.*;
import java.net.*;
public class Server {
  public static void main(String[] args) throws Exception {
    try (ServerSocket server = new ServerSocket(5000)) {
      System.out.println("Server waiting on port 5000...");
      try (Socket socket = server.accept();
           BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
           PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
        String msg = in.readLine();
        System.out.println("Client: " + msg);
        out.println("Message received: " + msg);
      }
    }
  }
}

// Client.java
import java.io.*;
import java.net.*;
public class Client {
  public static void main(String[] args) throws Exception {
    try (Socket socket = new Socket("localhost", 5000);
         BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
         BufferedReader keyboard = new BufferedReader(new InputStreamReader(System.in))) {
      System.out.print("Enter message: ");
      out.println(keyboard.readLine());
      System.out.println("Server: " + in.readLine());
    }
  }
}
```

Expected output / verification: Example: client enters “Hello”; server prints “Client: Hello”; client receives “Message received: Hello”.

Result: Java TCP client-server communication was established and a message was exchanged.

Viva: ServerSocket listens; Socket represents a connection; accept() waits for a client; TCP is reliable and ordered.

Q3. Network Diagnostic Commands

Aim: Run common network diagnostics and interpret their outputs.

Requirements: Windows Command Prompt/PowerShell; Linux terminal.

Theory: Diagnostic commands test reachability, show interface configuration, inspect routes/connections, and query DNS.

Procedure / Algorithm

15. Run ping 127.0.0.1 to test the local TCP/IP stack.
16. Run ping 8.8.8.8 to test IP reachability (internet/firewall conditions may affect results).
17. On Windows run ipconfig and ipconfig /all. Record IPv4 address, subnet mask, gateway, DNS server, and MAC/Physical Address.

18. On Linux run ip addr or ifconfig (if installed). Record interface address and status.
19. Run tracert example.com on Windows or traceroute example.com on Linux to view hops.
20. Run netstat -ano on Windows or ss -tuln on Linux to inspect connections/listening ports.
21. Run nslookup example.com to see the DNS server and resolved address.

Expected output / verification: Ping replies indicate reachability; “Request timed out” can indicate packet loss, filtering, or an unreachable host. ipconfig shows addressing; nslookup shows DNS results; traceroute shows path hops; netstat shows connection/port state.

Result: Network diagnostic commands were executed and their outputs interpreted.

Viva: What does ping use? ICMP Echo. What does DNS do? Maps names to IP addresses. Why can traceroute show timeouts? Routers may filter probe responses.

Q4. Star Topology Using a Switch in Packet Tracer

Aim: Build a star LAN, assign IP addresses, and test connectivity.

Requirements: Cisco Packet Tracer; 1 switch and 3 PCs.

Theory: A star topology connects each end device to a central switch.

Procedure / Algorithm

22. Place one 2960 switch and three PCs.
23. Connect each PC FastEthernet port to a switch port using Copper Straight-Through (or use automatic connection).
24. Configure PC0 as 192.168.1.10/24, PC1 as 192.168.1.11/24, and PC2 as 192.168.1.12/24. Leave gateway blank for this same-LAN test.
25. Wait for switch links to turn green.
26. Open PC0 → Desktop → Command Prompt; run ping 192.168.1.11 and ping 192.168.1.12.
27. Use Simulation mode or Add Simple PDU to observe packet movement; save the .pkt file.

Expected output / verification: Successful ping replies between PCs. First ping may time out while ARP resolves; repeat it.

Result: A switched star topology was created and connectivity verified.

Viva: Why use a switch? It forwards frames based on learned MAC addresses. A single cable failure usually affects only that endpoint.

Q5. CRC Error Detection in Java

Aim: Implement modulo-2 division for CRC and detect changed data.

Requirements: JDK.

Theory: CRC appends a remainder from modulo-2 binary division. XOR is used instead of subtraction; CRC detects many transmission errors but does not correct them.

Procedure / Algorithm

28. Enter a binary dataword and a binary generator polynomial.
29. Append generator-length minus one zero bits to the dataword.
30. Divide using XOR long division to calculate the remainder.
31. Append the remainder to the original dataword to form the codeword.
32. Divide the received codeword by the same generator; zero remainder means no error detected.

Program / Configuration

```
import java.util.*;
public class CRC {
  static String divide(String bits, String gen) {
    char[] a = bits.toCharArray();
    int n = gen.length();
    for (int i = 0; i <= a.length - n; i++) {
      if (a[i] == '1') {
        for (int j = 0; j < n; j++)
          a[i+j] = (a[i+j] == gen.charAt(j)) ? '0' : '1';
      }
    }
    return new String(a, a.length - (n - 1), n - 1);
  }
  static boolean binary(String s) { return s.matches("[01]+"); }
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Data bits: "); String data = sc.next();
    System.out.print("Generator bits: "); String gen = sc.next();
    if (!binary(data) || !binary(gen) || gen.charAt(0)!='1' || gen.charAt(gen.length()-1)!='1' || gen.length()<2)
{
      System.out.println("Use binary strings; generator must start and end with 1."); return;
    }
    String padded = data + "0".repeat(gen.length()-1);
    String rem = divide(padded, gen);
    String codeword = data + rem;
    System.out.println("CRC remainder: " + rem);
    System.out.println("Transmitted codeword: " + codeword);
    System.out.print("Received codeword (enter above codeword to test): ");
    String recv = sc.next();
    if (recv.length()!=codeword.length() || !binary(recv)) {
      System.out.println("Invalid received codeword."); return;
    }
    String check = divide(recv, gen);
    System.out.println("Syndrome/remainder: " + check);
    System.out.println(check.chars().allMatch(c -> c=='0') ? "No error detected" : "Error detected");
  }
}
```

Expected output / verification: Example input data 1101 and generator 1011. Program prints a CRC remainder and codeword; entering the unchanged codeword should produce an all-zero check remainder. Flip one bit and test again.

Result: CRC remainder and codeword were generated, and a received frame was checked.

Viva: Why XOR? Modulo-2 arithmetic has no carry/borrow. CRC detects errors, but a zero remainder does not mathematically guarantee that no error occurred.

Q6. Switch MAC Address Table

Aim: Observe how a switch learns MAC addresses while forwarding frames.

Requirements: Packet Tracer; one switch and two PCs.

Theory: A switch learns the source MAC address and incoming port of received frames, then uses its MAC table to forward known unicast frames.

Procedure / Algorithm

33. Place one switch and two PCs; connect each PC to the switch.
34. Set PC0 to 192.168.1.10/24 and PC1 to 192.168.1.11/24.
35. Ping PC1 from PC0 to generate traffic and allow ARP/MAC learning.

36. Click the switch → CLI. If prompted, press Enter; run enable then show mac address-table.
37. Observe dynamic entries mapping each PC MAC address to the connected switch port. Repeat after generating traffic if the table is empty.

Program / Configuration

```
Switch> enable
Switch# show mac address-table
! Typical command output includes VLAN, MAC address, type (DYNAMIC), and port.
```

Expected output / verification: The MAC table shows dynamically learned addresses on the ports used by the PCs.

Result: The switch learns from source MAC addresses, not destination addresses. Entries can age out after inactivity.

Q7. IP Addressing and Subnetting

Aim: Divide a network into subnets and verify device connectivity.

Requirements: Paper calculation; Packet Tracer or computers.

Theory: A subnet mask separates network and host bits. Example: 192.168.10.0/24 divided into four equal subnets requires borrowing 2 bits, producing /26 subnets.

Procedure / Algorithm

38. For a /24 network split into four equal subnets: 2^2 = 4 subnets, so new prefix is /26 and mask is 255.255.255.192.
39. Block size is 256 − 192 = 64 in the last octet.
40. Subnet 1: 192.168.10.0/26, usable hosts .1–.62, broadcast .63.
41. Subnet 2: 192.168.10.64/26, usable hosts .65–.126, broadcast .127.
42. Subnet 3: 192.168.10.128/26, usable hosts .129–.190, broadcast .191.
43. Subnet 4: 192.168.10.192/26, usable hosts .193–.254, broadcast .255.
44. Assign devices in the same subnet distinct usable addresses and the same mask. For different subnets, configure a router and correct default gateways.
45. Run ipconfig /all or ip addr to inspect settings; use ping to verify connectivity.

Expected output / verification: For a same-subnet test, two hosts such as 192.168.10.10/26 and 192.168.10.20/26 should communicate directly. Hosts in different /26 subnets require routing.

Result: Subnets, host ranges, masks, and connectivity were calculated and verified.

Viva: Usable hosts per /26 subnet = 2^6 − 2 = 62 for ordinary IPv4 subnets.

Q8. Star Topology Using a Hub

Aim: Create a hub-based star and observe frame delivery.

Requirements: Packet Tracer; one hub and three PCs.

Theory: A hub is a Physical-layer repeater that repeats incoming bits out other ports; unlike a switch, it does not learn MAC addresses.

Procedure / Algorithm

46. Place a hub and three PCs.
47. Connect each PC to a hub port with suitable copper Ethernet cables.
48. Assign 192.168.2.10/24, 192.168.2.11/24, and 192.168.2.12/24.
49. Wait for links to activate and ping between PCs.
50. Switch to Simulation mode and observe that hub traffic is repeated to other ports.

Expected output / verification: Ping succeeds between hosts in the same subnet; simulation illustrates hub flooding/repetition.

Result: A hub-based star topology was built and tested.

Viva: Hub vs switch: hub repeats bits to all other ports; switch forwards frames selectively using its MAC table.

Q9. Routing with RIP in Packet Tracer

Aim: Configure two routers to connect two different LANs using RIP v2.

Requirements: Packet Tracer; two routers, two switches, and two PCs.

Theory: RIP is a distance-vector routing protocol. RIP v2 supports classless routing and uses hop count; maximum usable hop count is 15.

Procedure / Algorithm

51. Build PC-A—Switch-A—R1—R2—Switch-B—PC-B. Use router interfaces supported by your router model; add serial modules if needed.
52. Example addressing: LAN-A 192.168.1.0/24; R1 LAN 192.168.1.1; PC-A 192.168.1.10 gateway 192.168.1.1.
53. Router link: R1 10.0.0.1/30 and R2 10.0.0.2/30. LAN-B 192.168.2.0/24; R2 LAN 192.168.2.1; PC-B 192.168.2.10 gateway 192.168.2.1.
54. Configure each interface with IP address and no shutdown. For serial links, set clock rate on the DCE end if required.
55. On R1 configure router rip, version 2, no auto-summary, network 192.168.1.0, network 10.0.0.0.
56. On R2 configure router rip, version 2, no auto-summary, network 192.168.2.0, network 10.0.0.0.
57. Run show ip route and ping PC-B from PC-A. Use show ip protocols if troubleshooting.

Program / Configuration

```
! Example R1 (adjust interface names to your router)
enable
configure terminal
interface gigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 no shutdown
exit
interface gigabitEthernet0/1
 ip address 10.0.0.1 255.255.255.252
 no shutdown
exit
router rip
 version 2
 no auto-summary
 network 192.168.1.0
 network 10.0.0.0
end
show ip route
```

Expected output / verification: Routing table should learn the remote LAN as an RIP route (marked R); end-to-end ping should succeed after convergence.

Result: RIP uses hop count. OSPF is a link-state protocol and generally converges faster in larger networks. Network statements and interface names must match the actual topology.

Q10. Stop-and-Wait ARQ in Java

Aim: Simulate sending one frame at a time and receiving acknowledgements.

Requirements: JDK.

Theory: Stop-and-Wait ARQ sends one frame and waits for an ACK before sending the next. If a frame/ACK is lost, a timeout causes retransmission.

Procedure / Algorithm

58. Set the total number of frames.
59. For each frame, simulate transmission.
60. Simulate ACK reception; if ACK is missing, retransmit the same frame.
61. Continue only after ACK is received.

Program / Configuration

```
import java.util.*;
public class StopWait {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Number of frames: ");
    int n = sc.nextInt();
    Random random = new Random();
    for (int i = 1; i <= n; i++) {
      boolean ack;
      do {
        System.out.println("Sending frame " + i);
        // Randomly simulate ACK loss for demonstration
        ack = random.nextInt(4) != 0;
        if (ack) System.out.println("ACK received for frame " + i);
        else System.out.println("ACK lost/timeout. Retransmitting frame " + i);
      } while (!ack);
    }
    System.out.println("All frames transmitted successfully.");
  }
}
```

Expected output / verification: Output shows frame transmission, ACK received messages, and possible retransmissions; random simulation output varies on each run.

Result: Stop-and-Wait ARQ was simulated with acknowledgements and retransmission.

Viva: Why wait? It ensures the sender does not advance before confirmation. Sequence numbers help distinguish retransmissions and duplicate frames.

Q11. Capture and Analyze Packets Using Wireshark

Aim: Capture packets and identify protocol fields.

Requirements: Wireshark installed; permission to capture traffic.

Theory: Wireshark captures and decodes packets. A display filter narrows the packets shown without deleting captured packets.

Procedure / Algorithm

62. Open Wireshark and select the active network interface showing traffic.
63. Click Start Capture.
64. Generate traffic: open a website or run ping to a permitted host.
65. Stop capture after several packets appear.
66. Use display filters such as dns, icmp, tcp, udp, or http. HTTPS traffic usually exposes metadata but not encrypted page contents.
67. Select a packet and expand Ethernet, Internet Protocol, TCP/UDP, and application protocol sections.

68. Record source/destination MAC and IP addresses, protocol, ports where applicable, packet length, and TCP flags if present.

Expected output / verification: For a ping, look for ICMP Echo request and Echo reply. For a DNS lookup, look for DNS query and response. Actual packets depend on the traffic generated.

Result: Packets were captured and protocol headers were interpreted.

Viva: Capture filter acts while capturing; display filter filters the current packet list. HTTPS content is encrypted by TLS.

Q12. Design Any Topology and Test Connectivity

Aim: Create a topology of choice and verify connectivity.

Requirements: Packet Tracer; PCs and a switch (star is simplest).

Theory: Topology describes how devices are connected physically or logically.

Procedure / Algorithm

69. Choose Star topology for a straightforward demonstration.
70. Place one switch and three PCs; connect each PC to the switch.
71. Assign 192.168.5.10/24, 192.168.5.11/24, and 192.168.5.12/24.
72. Verify that all interfaces are up and all PCs share the same subnet.
73. Ping each PC from PC0; optionally use Add Simple PDU in Simulation mode.
74. Save the topology and capture a screenshot.

Expected output / verification: All same-subnet pings should succeed after ARP learning.

Result: The selected network topology was designed and tested.

Viva: Star uses a central device; bus uses a shared backbone; ring forms a loop; mesh offers multiple paths but needs more links.

Q13. DHCP Server in Packet Tracer

Aim: Configure automatic IPv4 address allocation using DHCP.

Requirements: Packet Tracer; one server, one switch, and two PCs.

Theory: DHCP dynamically leases IP settings to clients. The common exchange is DORA: Discover, Offer, Request, Acknowledge.

Procedure / Algorithm

75. Connect Server0 and two PCs to a switch.
76. Set Server0 static IP to 192.168.10.2, subnet mask 255.255.255.0.
77. Open Server0 → Services → DHCP and switch the service On.
78. Create a pool named LAN; default gateway 192.168.10.1 if a router is present; DNS server 192.168.10.2 or another valid DNS IP; start IP 192.168.10.100; subnet mask 255.255.255.0; set maximum users, then Add/Save.
79. If no router is used, leave gateway unused/0.0.0.0 if accepted. Ensure the pool does not overlap the server's static address.
80. On each PC open Desktop → IP Configuration → DHCP.
81. Check assigned addresses and run ipconfig; ping between clients.

Expected output / verification: Clients receive distinct addresses from the configured pool. If a gateway is configured, it must actually exist and be connected.

Result: DHCP dynamically assigned IP configuration to clients.

Viva: DORA means Discover, Offer, Request, Acknowledge. DHCP reduces manual configuration errors.

Q14. TCP Daytime Server in Java

Aim: Send current date and time from a server to a client.

Requirements: JDK; two terminals.

Theory: The server accepts a TCP connection and writes a time string to the connected client.

Procedure / Algorithm

82. Save the following as DaytimeServer.java.
83. Compile using javac DaytimeServer.java.
84. Run java DaytimeServer in one terminal.
85. Connect from another terminal with a Java client or a TCP utility; the server sends the current date/time and closes the connection.

Program / Configuration

```
// DaytimeServer.java
import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
public class DaytimeServer {
  public static void main(String[] args) throws IOException {
    try (ServerSocket server = new ServerSocket(1313)) {
      System.out.println("Daytime server listening on port 1313");
      while (true) {
        try (Socket client = server.accept();
             PrintWriter out = new PrintWriter(client.getOutputStream(), true)) {
          out.println("Current date and time: " + LocalDateTime.now());
        }
      }
    }
  }
}

// Optional client: DaytimeClient.java
import java.io.*;
import java.net.*;
public class DaytimeClient {
  public static void main(String[] args) throws IOException {
    try (Socket s = new Socket("localhost", 1313);
         BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()))) {
      System.out.println(in.readLine());
    }
  }
}
```

Expected output / verification: Run the server, then compile/run DaytimeClient. The client prints the server's current date and time.

Result: The TCP Daytime server delivered a time string to a client.

Viva: ServerSocket listens for incoming connections. The service here uses a custom port 1313, not the traditional daytime port 13.

Q15. Compare OSI and TCP/IP Models

Aim: Compare layers, protocols, and encapsulation in OSI and TCP/IP.

Requirements: Networking notes.

Theory: OSI is a seven-layer reference model; the commonly taught TCP/IP model has four layers.

Procedure / Algorithm

86. Write the OSI layers: Application, Presentation, Session, Transport, Network, Data Link, Physical (top to bottom).
87. Write TCP/IP layers: Application, Transport, Internet, Network Access/Link.
88. Map OSI Application + Presentation + Session to TCP/IP Application.
89. Map OSI Transport to TCP/IP Transport; OSI Network to Internet; OSI Data Link + Physical to Network Access.
90. Explain encapsulation: data is wrapped with transport, IP, and link headers; receiver removes headers in reverse.
91. Give protocol examples: HTTP/DNS at Application, TCP/UDP at Transport, IP/ICMP at Internet, Ethernet/Wi-Fi at Link.

Expected output / verification: Present a neat comparison table in the lab file and explain encapsulation using a web request.

Result: The models, layer mapping, and encapsulation were compared.

Viva: OSI has seven layers; TCP/IP is commonly represented as four layers, although some texts use five.

Q16. Basic Ethernet Network in Packet Tracer

Aim: Configure an Ethernet LAN and verify connectivity.

Requirements: Packet Tracer; switch and PCs.

Theory: Ethernet is a family of LAN technologies. A switch connects devices within a LAN using Ethernet frames.

Procedure / Algorithm

92. Place one switch and two PCs.
93. Connect PCs to switch Ethernet ports with straight-through cables.
94. Set PC0 to 192.168.20.10/24 and PC1 to 192.168.20.11/24.
95. Wait for green links.
96. From PC0, run ping 192.168.20.11.
97. Inspect Simulation mode to observe ARP and ICMP traffic; save the .pkt file.

Expected output / verification: Successful ICMP Echo replies confirm same-LAN connectivity.

Result: A basic Ethernet LAN was configured and tested.

Viva: An Ethernet frame contains source/destination MAC addresses, type/length, payload, and frame check sequence.

Q17. DNS and Web Server in Packet Tracer

Aim: Configure DNS name resolution and access a hosted web page.

Requirements: Packet Tracer; server, switch, PC.

Theory: DNS maps a domain name to an IP address. HTTP serves web content from a web server.

Procedure / Algorithm

98. Connect Server0 and PC0 to a switch.

99. Set Server0 static IP 192.168.30.2/24 and PC0 192.168.30.10/24.
100. On Server0 → Services → HTTP, turn HTTP On and edit the default page if desired.
101. On Server0 → Services → DNS, turn DNS On and add an A record: Name www.lab.local, Address 192.168.30.2; click Add.
102. On PC0, set DNS server to 192.168.30.2 in Desktop → IP Configuration.
103. Open PC0 → Desktop → Web Browser and enter http://www.lab.local.
104. If name resolution fails, test ping 192.168.30.2 and confirm the DNS record and client DNS setting.

Expected output / verification: The browser displays the hosted page using the domain name. Direct IP access may work even if DNS is misconfigured.

Result: DNS resolution and web server access were demonstrated.

Viva: DNS typically uses port 53; HTTP uses port 80; HTTPS commonly uses port 443.

Q18. Sliding Window Protocol in Java

Aim: Simulate sending multiple frames before acknowledgements arrive.

Requirements: JDK.

Theory: Sliding Window allows a sender to have several unacknowledged frames in flight, improving link utilization compared with Stop-and-Wait.

Procedure / Algorithm

105. Set the total number of frames and sender window size.
106. Send frames in batches no larger than the window.
107. Display ACKs for the transmitted frames and slide the window forward.
108. Continue until all frames are acknowledged.

Program / Configuration

```
import java.util.Scanner;
public class SlidingWindow {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Total frames: "); int total = sc.nextInt();
    System.out.print("Window size: "); int w = sc.nextInt();
    if (total < 1 || w < 1) { System.out.println("Values must be positive."); return; }
    int base = 1;
    while (base <= total) {
      int end = Math.min(base + w - 1, total);
      System.out.println("\nWindow: frames " + base + " to " + end);
      for (int i = base; i <= end; i++) System.out.println("Sending frame " + i);
      for (int i = base; i <= end; i++) System.out.println("ACK received for frame " + i);
      base = end + 1;
    }
    System.out.println("\nAll frames acknowledged.");
  }
}
```

Expected output / verification: For total frames 7 and window size 3, frames 1–3 are sent, acknowledged, then 4–6, then 7.

Result: Sliding Window transmission and acknowledgement were simulated.

Viva: Go-Back-N and Selective Repeat are sliding-window ARQ variants. The sample program models successful ACKs rather than packet loss.

Q19. CRC Error Detection — Test Cases

Aim: Test CRC with valid and altered received codewords.

Requirements: JDK; CRC program from Question 5.

Theory: CRC verification divides the received codeword by the generator polynomial; a nonzero remainder indicates a detected error.

Procedure / Algorithm

109. Run the CRC program from Question 5.
110. Enter data bits 1101 and generator 1011.
111. Copy the generated codeword and enter it unchanged as the received codeword.
112. Confirm the check remainder is all zeros.
113. Run again and flip one bit in the codeword; enter the altered codeword.
114. Confirm a nonzero remainder is reported for the chosen test.

Expected output / verification: Valid codeword → zero remainder; altered codeword → commonly nonzero remainder. Some error patterns can be undetected, so avoid claiming CRC detects every possible error.

Result: CRC was tested using valid and modified received data.

Viva: CRC is an error-detection technique. The generator polynomial determines which error patterns can be detected.

Q20. Basic Firewall or VPN Scenario

Aim: Demonstrate a basic access-control rule and verify permitted/blocked traffic.

Requirements: Packet Tracer router/ACL-capable device or a lab firewall; use only an authorized lab network.

Theory: A firewall filters traffic according to rules. In Packet Tracer, an extended ACL can demonstrate filtering; this is a basic ACL example, not a full VPN deployment.

Procedure / Algorithm

115. Build PC-A—Router—PC-B on two networks: PC-A LAN 192.168.1.0/24, PC-B LAN 192.168.2.0/24.
116. Configure router interfaces as 192.168.1.1/24 and 192.168.2.1/24, and PCs with their respective gateways.
117. Verify ping works before applying the ACL.
118. Create an extended ACL to deny ICMP from PC-A to PC-B and permit other IP traffic.
119. Apply the ACL inbound on the interface facing PC-A; test ping again and inspect ACL counters with show access-lists.
120. Remove the ACL or change the rule to permit ICMP, then verify ping works again.

Program / Configuration

```
! Example Cisco IOS ACL; adjust interface names and addresses
enable
configure terminal
access-list 101 deny icmp 192.168.1.0 0.0.0.255 192.168.2.0 0.0.0.255
access-list 101 permit ip any any
interface gigabitEthernet0/0
 ip access-group 101 in
end
show access-lists
show ip interface gigabitEthernet0/0
```

Expected output / verification: Ping should fail when the deny rule is active, while other IP traffic is permitted by the second rule. ACLs are processed top to bottom.

Result: An access-control firewall scenario was configured and verified.

Viva: ACL wildcard mask 0.0.0.255 corresponds to a /24 network. A VPN additionally creates an encrypted tunnel; do not claim an ACL alone is a VPN.

Q21. TCP Echo Server in Java

Aim: Receive a client message and send the same message back.

Requirements: JDK; two terminals.

Theory: A TCP echo server reads a line from a client and returns the identical line.

Procedure / Algorithm

121. Save as EchoServer.java and EchoClient.java in the same folder.
122. Compile using javac EchoServer.java EchoClient.java.
123. Run java EchoServer first, then java EchoClient in another terminal.
124. Type a message in the client; verify that the server prints it and the client receives the same text.

Program / Configuration

```
// EchoServer.java
import java.io.*;
import java.net.*;
public class EchoServer {
  public static void main(String[] args) throws IOException {
    try (ServerSocket server = new ServerSocket(6000)) {
      System.out.println("Echo server listening on 6000");
      while (true) {
        try (Socket s = server.accept();
             BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
             PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {
          String msg;
          while ((msg = in.readLine()) != null) {
            out.println(msg);
            if (msg.equalsIgnoreCase("bye")) break;
          }
        }
      }
    }
  }
}

// EchoClient.java
import java.io.*;
import java.net.*;
public class EchoClient {
  public static void main(String[] args) throws IOException {
    try (Socket s = new Socket("localhost", 6000);
         BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
         PrintWriter out = new PrintWriter(s.getOutputStream(), true);
         BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
      String msg;
      do {
        System.out.print("Message (bye to exit): ");
        msg = kb.readLine();
        out.println(msg);
        System.out.println("Echo: " + in.readLine());
      } while (!msg.equalsIgnoreCase("bye"));
    }
  }
}
```

Expected output / verification: Client input “Hello Network” returns “Echo: Hello Network”. Type bye to end.

Result: The TCP echo server received and returned client messages.

Viva: An echo server is useful for testing connectivity. TCP provides a byte stream, so applications define message boundaries (this example uses newline-delimited lines).

Q22. IPC Message Queue in Java

Aim: Exchange messages between producer and consumer threads using a queue.

Requirements: JDK.

Theory: Inter-process communication (IPC) lets independent execution units exchange data. Java BlockingQueue is a thread-safe queue suitable for demonstrating producer-consumer message passing within one JVM; it is not an OS-level queue shared across unrelated Java processes.

Procedure / Algorithm

125. Create a bounded BlockingQueue of strings.
126. Producer thread places messages into the queue.
127. Consumer thread takes messages from the queue and displays them.
128. Use a poison-pill message to signal the consumer to stop.

Program / Configuration

```
import java.util.concurrent.*;
public class MessageQueueIPC {
  public static void main(String[] args) throws InterruptedException {
    BlockingQueue<String> queue = new ArrayBlockingQueue<>(5);
    String STOP = "STOP";
    Thread producer = new Thread(() -> {
      try {
        queue.put("Message 1");
        queue.put("Message 2");
        queue.put("Message 3");
        queue.put(STOP);
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    Thread consumer = new Thread(() -> {
      try {
        while (true) {
          String msg = queue.take();
          if (msg.equals(STOP)) break;
          System.out.println("Received: " + msg);
        }
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
    producer.start(); consumer.start();
    producer.join(); consumer.join();
    System.out.println("Communication completed.");
  }
}
```

Expected output / verification: Output includes Received: Message 1, Message 2, Message 3, then Communication completed.

Result: Producer-consumer message exchange was implemented using a thread-safe queue.

Viva: BlockingQueue blocks when full/empty as needed. If the examiner specifically requires separate OS processes, clarify that this example is thread-based IPC-style communication and ask which cross-process mechanism is expected.

Q23. Subnetting and Diagnostics for a Small Network

Aim: Configure IP addresses and subnetting, then verify using diagnostic commands.

Requirements: Two PCs and a switch in Packet Tracer, or Windows/Linux computers.

Theory: Hosts in the same IP subnet can communicate directly; hosts in different subnets need a router and gateway.

Procedure / Algorithm

129. Use network 192.168.40.0/24 and split into two /25 subnets. Mask is 255.255.255.128; block size is 128.
130. Subnet A: 192.168.40.0/25, usable .1–.126, broadcast .127. Subnet B: 192.168.40.128/25, usable .129–.254, broadcast .255.
131. Assign PC-A 192.168.40.10/25 and PC-B 192.168.40.20/25 to test same-subnet connectivity.
132. Run ipconfig /all on Windows or ip addr on Linux to inspect IP/mask.
133. Run ping 192.168.40.20; use tracert/traceroute if testing a routed destination and nslookup to test DNS.
134. If testing communication across the two /25 subnets, add a router with one interface in each subnet and set each PC's default gateway.

Expected output / verification: Check addresses and masks first; same-subnet ping should work. Cross-subnet ping requires router interfaces, gateways, and routes.

Result: Subnetting and IP connectivity were configured and verified with diagnostic commands.

Viva: A /25 has 7 host bits, so 2^7 − 2 = 126 usable hosts in a conventional subnet.

Q24. Basic SDN Topology Using Mininet

Aim: Create a simple SDN emulated network and test host connectivity.

Requirements: Linux environment (often Ubuntu/VM), Mininet, Open vSwitch, and an SDN controller if required by the lab.

Theory: Software-Defined Networking separates the control plane from the data plane. Mininet emulates hosts, switches, and links on one machine.

Procedure / Algorithm

135. Use an authorized Linux lab/VM. Install Mininet using your instructor's recommended package method; package availability differs by distribution.
136. Run a simple topology: sudo mn --topo single,3 --mac --controller=default.
137. At the Mininet prompt, run nodes, net, and dump to inspect the topology.
138. Run pingall to test all host pairs.
139. Run h1 ping -c 3 h2 for a specific test.
140. Exit using exit, then clean stale Mininet state with sudo mn -c if necessary.
141. For a controller-based demonstration, use the controller IP/port specified by your lab and confirm it is running before starting Mininet.

Program / Configuration

```
# Common Mininet commands (run in a Linux terminal)
sudo mn --topo single,3 --mac --controller=default
# At mininet> prompt:
nodes
net
dump
pingall
h1 ping -c 3 h2
exit
sudo mn -c
```

Expected output / verification: pingall should report connectivity between all emulated hosts when the topology/controller is functioning.

Result: The Mininet topology was created and host connectivity tested.

Viva: SDN separates control and forwarding functions. Mininet is an emulator, not a physical network. Commands and controller setup can vary by Linux distribution.

Quick practical checklist

- For every code task: save the correct filename, compile, run, test normal input, and explain the key functions.
- For every Packet Tracer task: check cable/port status, IP address, subnet mask, gateway where needed, services, and routing.
- For connectivity: test in order—link status → local IP settings → same-subnet ping → gateway → remote network → DNS/service.
- For CRC: demonstrate both an unchanged codeword and a modified codeword; explain that CRC detects but does not correct errors.
- For ARQ: explain ACK, timeout, retransmission, and sequence numbers.
- For lab-file marks: include a topology/program screenshot, observed output, result, and conclusion.
- Do not claim a test passed until you have run it on your own system; interface names and Packet Tracer device models may differ.

Short viva revision

- **IP address:** Logical address used to identify an interface on an IP network.
- **MAC address:** Link-layer address used for local Ethernet/Wi-Fi delivery.
- **Switch:** Learns source MAC addresses and forwards frames within a LAN.
- **Router:** Forwards packets between IP networks.
- **TCP:** Connection-oriented, reliable, ordered byte-stream transport.
- **UDP:** Connectionless datagram transport with no built-in delivery guarantee.
- **DNS:** Resolves domain names to IP addresses and other records.
- **DHCP:** Automatically leases IP configuration to clients.
- **CRC:** Error-detection technique based on modulo-2 division.
- **ACK:** Acknowledgement that confirms receipt of a frame/data.
- **Subnet mask:** Identifies network and host portions of an IPv4 address.
- **Firewall:** Enforces traffic rules between security zones or interfaces.
- **Wireshark:** Captures and analyzes network packets.
- **SDN:** Network architecture that separates control logic from packet forwarding.